

BFS — Breadth-First Search

Explication et fonctionnement dans le projet NAO

1. Qu'est-ce que le BFS ?

BFS signifie **Breadth-First Search**, ou **Recherche en Largeur** en français. C'est un algorithme de parcours de graphe ou de grille qui explore les cases voisines niveau par niveau, en s'éloignant progressivement du point de départ.

Contrairement à un algorithme en profondeur (DFS) qui s'engage à fond dans une seule direction avant de revenir en arrière, le BFS explore **toutes les directions en même temps**, comme une vague qui se propage depuis le point de départ.

2. Où est-il dans le code ?

Dans le projet NAO, le BFS se trouve dans la fonction `suivre_chemin_scene()` du fichier `scene_idc.py`. Son rôle est de trouver le chemin le plus court entre la position initiale du robot (N) et l'objectif (■), en ne circulant que sur les cases marquées ■.

```
def suivre_chemin_scene():
    start = (NAO[0], NAO[1])
    but = tuple(trouver_position("■"))
    queue = deque([start, [start]])
    visite = {start}

    while queue:
        (x, y), chemin_actuel = queue.popleft()
        if (x, y) == but:
            return [[p[0], p[1]] for p in chemin_actuel]
        for dx, dy in [(0,1), (0,-1), (1,0), (-1,0)]:
            nx, ny = x + dx, y + dy
            if (nx, ny) not in visite and 0 <= ny < len(scene):
                if scene[ny][nx] in ("■", "■"):
                    visite.add((nx, ny))
                    queue.append(((nx, ny), chemin_actuel + [(nx, ny)]))
    return [list(start)]
```

3. Comment fonctionne-t-il exactement ?

3.1 Initialisation

On crée une **file d'attente** (queue) contenant le point de départ N, ainsi qu'un ensemble **visite** pour ne pas repasser deux fois sur la même case.

```
queue = deque([ (start, [start]) ])
visite = {start}
```

3.2 Boucle principale

À chaque tour de boucle, on retire la première case de la queue. Si c'est l'objectif ■, on retourne le chemin parcouru. Sinon, on regarde les 4 cases adjacentes (haut, bas, gauche, droite). Pour chaque voisin accessible (■ ou ■) non encore visité, on l'ajoute à la queue avec le chemin mis à jour.

3.3 Exemple pas à pas sur la scène du projet

X	X	X	X	X	X	X
E	N	1	1	1	1	X
X	■	X	■	■	■	X
X	■	X	■	X	X	X
X	■	■	■	■	■	X
X	X	X	X	X	X	X

La scène du projet NAO — N=départ (bleu), ■=objectif (vert), ■=itinéraire (orange)

Étape 1 — Initialisation

```
queue = [ (N[1,1], chemin=[N]) ] | visite = { [1,1] }
```

Étape 2 — On sort N[1,1], on regarde ses voisins

```
bas [1,2]=■ → ajouté
haut [1,0]=X → bloqué
droite [2,1]=1 → ignoré (pas ■)
gauche [0,1]=E → ignoré
```

```
queue = [ ([1,2], chemin=[N, [1,2]]) ]
```

Étape 3 — On sort [1,2]

```
bas [1,3]=■ → ajouté
haut [1,1]=N → déjà visité
droite [2,2]=X → bloqué
gauche [0,2]=X → bloqué
```

```
queue = [ ([1,3], chemin=[N, [1,2], [1,3]]) ]
```

Étapes 4-5 — On continue vers [1,3] puis [1,4]

```
Le BFS descend le long de la colonne 1 jusqu'en [1,4].
queue = [ ([2,4], chemin=[N, [1,2], [1,3], [1,4], [2,4]]) ]
```

Étape 6 — Fourche en [3,4] : deux voisins ■

```
haut [3,3]=■ → ajouté (chemin de 7 cases)
```

droite [4,4]=■ → ajouté (chemin de 7 cases)

```
queue = [  
  ([3,3], chemin=[...,[3,4],[3,3]]), ← chemin long  
  ([4,4], chemin=[...,[3,4],[4,4]]) ← chemin court  
]
```

Résultat final

Le BFS explore les deux branches en parallèle.
Il atteint ■[5,4] depuis [4,4] en 8 étapes.
C'est forcément le plus court car le BFS
explore d'abord tous les chemins de longueur N
avant ceux de longueur N+1.

Chemin optimal : N→[1,2]→[1,3]→[1,4]→[2,4]→[3,4]→[4,4]→■

4. Pourquoi le BFS garantit le chemin optimal ?

Imagine le BFS comme une **vague qui se propage** depuis N :

Vague	Cases explorées	Distance depuis N
1	[1,2]	1 case
2	[1,3]	2 cases
3	[1,4]	3 cases
4	[2,4]	4 cases
5	[3,4]	5 cases
6	[3,3] et [4,4]	6 cases
7	[3,2] et [5,4]=■	7 cases

La première fois que la vague atteint ■ (vague 7), c'est forcément par le chemin le plus court.

Comme toutes les cases à distance N sont explorées avant celles à distance N+1, la première fois qu'on atteint ■, on est certain d'avoir trouvé le chemin le plus court. Si un chemin plus long existait, il aurait été découvert plus tard dans la queue.

5. Résumé

Concept	Explication
BFS	Breadth-First Search — Recherche en Largeur
Queue (file)	Structure FIFO : premier entré, premier sorti
Visite	Ensemble des cases déjà explorées pour éviter les boucles
Optimal	Le premier chemin trouvé est toujours le plus court
Cases valides	Uniquement ■ et ■ — les autres sont ignorées